

DAY 11 – REST API FUNDAMENTALS

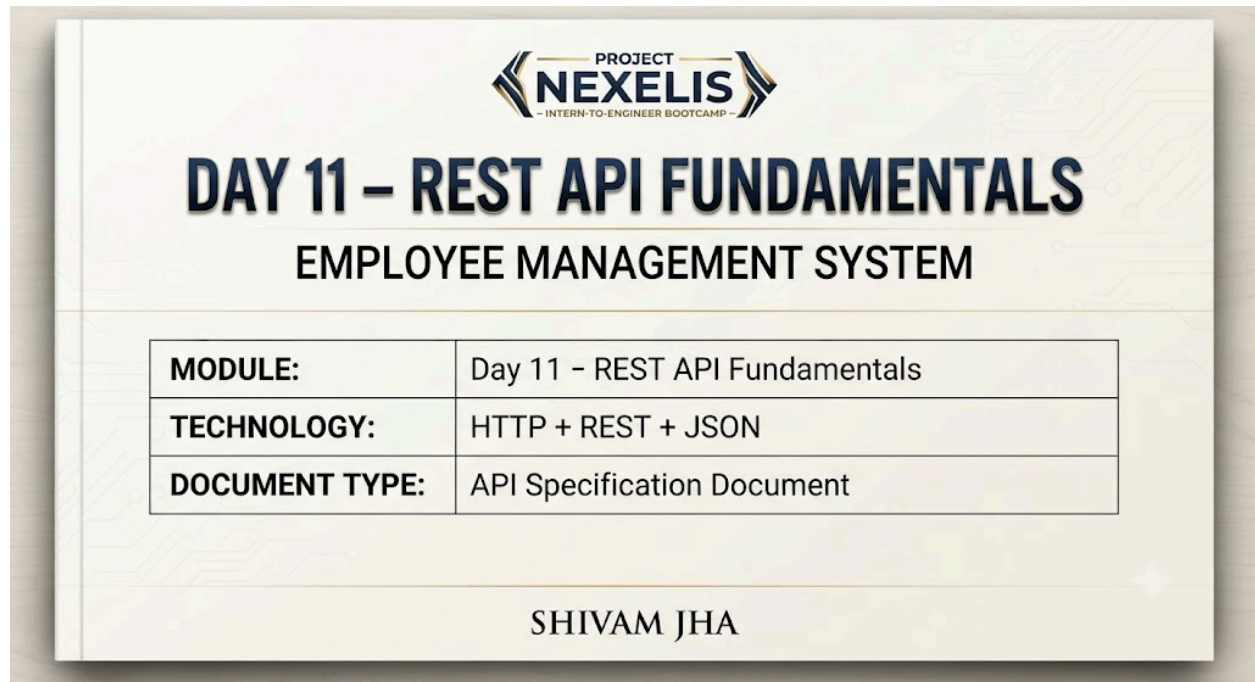


Table of Contents:-

1. Project Objective & Business Context
2. REST API & API Architecture
3. HTTP Fundamentals
4. HTTP Request
5. HTTP Response
6. JSON Data Format
7. Headers
8. URL & API Resource Design
9. Query Parameters

10. **Path Parameters**
11. **Request Body**
12. **HTTP Methods**
 - GET
 - POST
 - PUT
 - PATCH
 - DELETE
13. **HTTP Status Codes**
14. **Employee API Resource Design**

1. PROJECT OBJECTIVE & BUSINESS CONTEXT

The objective of **Day 11 – REST API Fundamentals** is to understand how a client application communicates with a backend system through a **standardized REST API interface**.

In this task, the Employee Management System is exposed as a set of **RESTful endpoints** that allow clients to create, retrieve, update, and delete employee records using **HTTP methods, JSON data, request/response structures, and appropriate status codes**.

The focus is to understand the complete communication flow:

Client



HTTP Request



REST API



Validation & Processing



Employee Data



HTTP Response



Client

Key Objectives

- Understand the fundamentals of **HTTP and REST APIs**.
- Understand how **requests and responses** work.
- Learn how **URLs, headers, parameters, and JSON bodies** carry information.
- Understand the purpose of **GET, POST, PUT, PATCH, and DELETE**.
- Use appropriate **HTTP status codes** for successful and failed operations.
- Design clear and consistent **Employee REST endpoints**.
- Define standardized **request and response JSON structures**.
- Handle **validation and error responses** consistently.
- Establish a clean API contract between the client and backend.

2. REST API & API Architecture

A **REST API (Representational State Transfer Application Programming Interface)** is an interface that allows different applications to communicate with backend resources using standard **HTTP methods and resource-based URLs**.

In our project, the Employee is the primary resource exposed through the API.

REST API = A standardized interface through which a client interacts with backend resources.

Key Points

- REST APIs expose **resources**, not database tables directly.
- Resources are identified using **URLs**.
- HTTP methods define the operation.
- JSON is commonly used for data exchange.

- Requests and responses follow a predictable structure.
- The API acts as a controlled boundary between client and backend.
- REST APIs are generally **stateless**: each request contains the information required to process it.
- The API should not expose direct database access to the client.

Real-Life Analogy

Think of an API as a **reception desk in a company**.

Employee/Visitor

|



Reception Desk

|



Relevant Department

|



Information / Action

The visitor does not directly enter every department.

Similarly:

Frontend



API



Backend Logic



Database

The API controls how the client interacts with the system.

How REST Is Used in Our Project

Endpoint Design

Operation	Method	Endpoint	Purpose
List employees	GET	/employees	Retrieve employees
Get employee	GET	/employees/{id}	Retrieve one employee
Create employee	POST	/employees	Create a new employee
Replace employee	PUT	/employees/{id}	Replace employee data
Update employee	PATCH	/employees/{id}	Partially update employee

Delete employee	DELETE E	/employees /{id}	Remove employee
--------------------	-------------	---------------------	-----------------

I designed the Employee Management API using a resource-oriented REST approach. The employee resource is exposed through predictable URLs, while HTTP methods define the operation. The API acts as a controlled boundary between the client and PostgreSQL, handling validation, business processing, and standardized JSON responses..

3. HTTP Fundamentals

HTTP (HyperText Transfer Protocol) is the standard communication protocol used between a client and a server.

In our Employee Management System, HTTP carries the request from the client to the REST API and the response from the REST API back to the client.

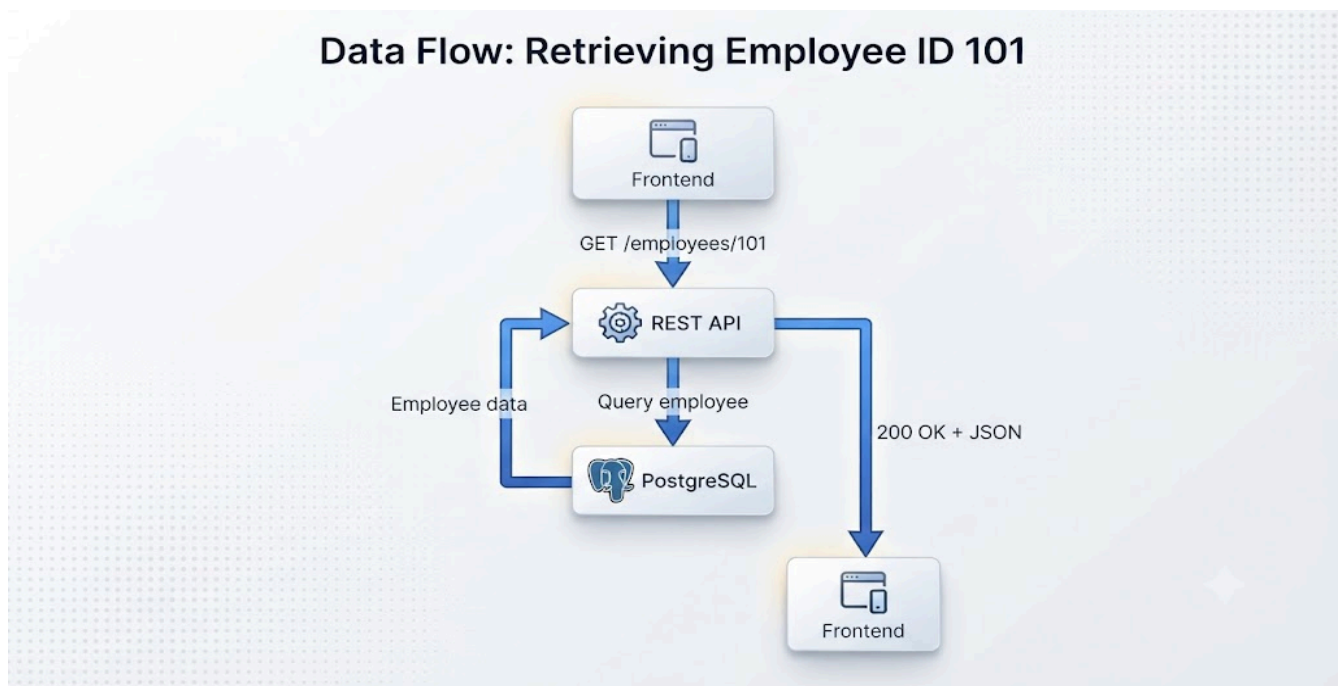
Key Points

- HTTP enables **client-server communication**.
- Every interaction is based on a **request and response**.
- A request contains information such as:
 - Method
 - URL
 - Headers
 - Body, when required
- A response contains:
 - Status code
 - Headers
 - Response body
- HTTP methods communicate the intended operation.

- Status codes communicate the result of the operation.
- HTTP is **stateless**: each request is processed independently, with the required context supplied by the request.

HTTP Request–Response Cycle

A typical Employee API operation works like this:



For our Employee API:

Status

Usage

200

Employee successfully
retrieved/updated

201	Employee successfully created
204	Employee successfully deleted
400	Malformed or invalid request
404	Employee does not exist
409	Duplicate email/business conflict
422	Request validation failed
500	Unexpected server-side failure

HTTP Is Not the Same as REST

This is an important distinction.

HTTP



Communication Protocol

REST



Architectural Style

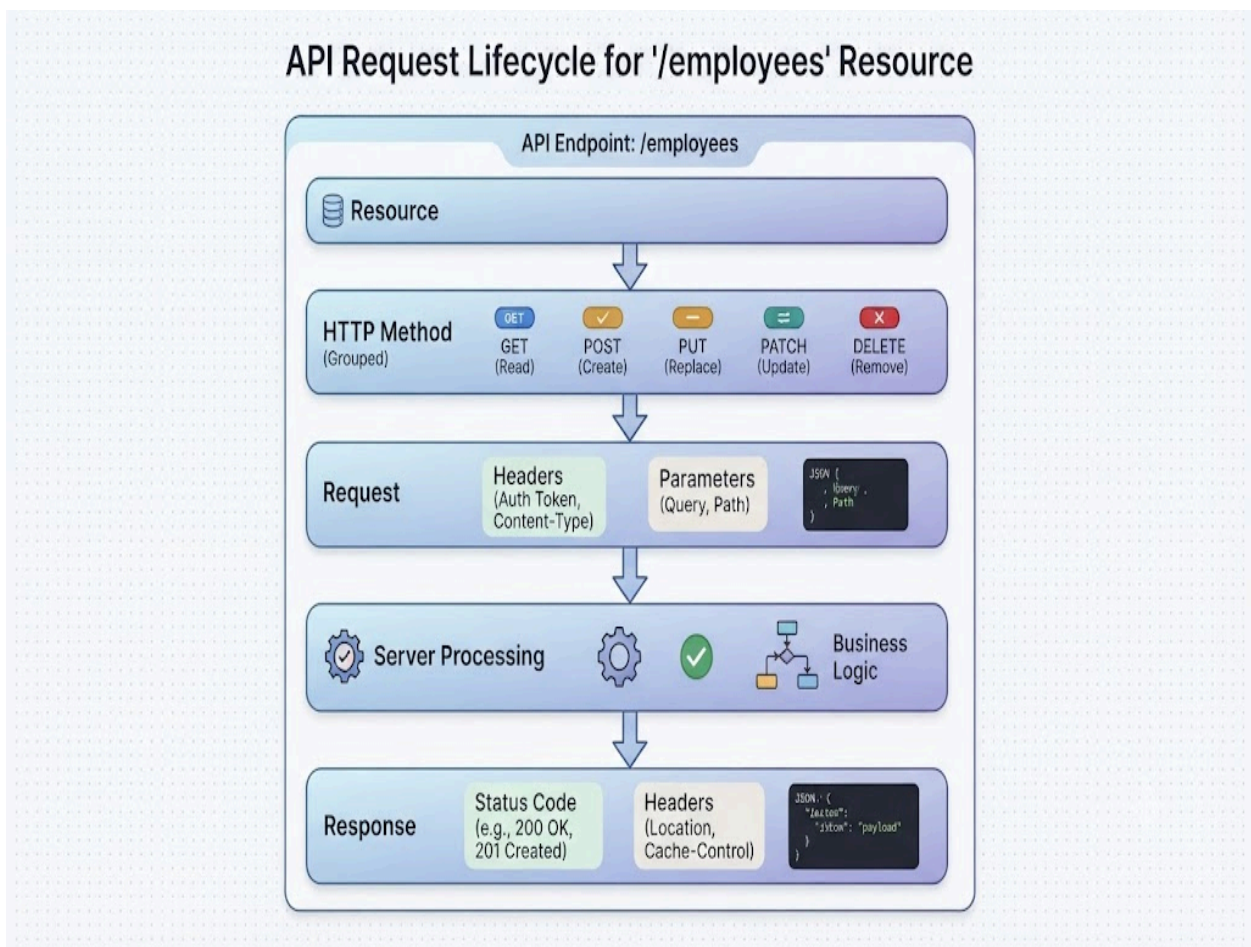


How we design APIs using resources and HTTP semantics

HTTP defines how communication happens; REST defines how we structure the API around resources and operations.

HTTP in Our Employee Project

Our Day 11 API follows:



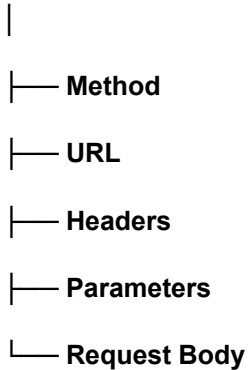
4. HTTP Request

An **HTTP Request** is the message sent by a client to a server/API to tell it **what operation is required and what data is needed**.

4.1 Request Structure

A typical HTTP request contains:

HTTP Request



4.2 Method

The method defines **what the client wants to do**.

Method	Purpose
GET	Retrieve data
POST	Create new data
PUT	Replace/update a resource
PATCH	Partially update a resource

DELETE Delete a resource

Example:

GET /employees

means:

“Give me the employee records.”

4.3 URL

The URL identifies **which API resource the client wants to access**.

GET /employees/101

Here:

- /employees → Employee resource
- /101 → Specific employee
- GET → Retrieve operation

4.4 Headers

Headers provide **additional information about the request**.

Example:

Content-Type: application/json

Accept: application/json

Authorization: Bearer <token>

Common headers:

- Content-Type → format of request body
- Accept → response format expected by client
- Authorization → authentication credentials

4.5 Parameters

Parameters provide additional information to the API.

Two important types:

Path parameter

GET /employees/101

101 identifies the specific employee.

Query parameter

GET /employees?department=IT

department=IT filters the employee list.

4.6 Request Body

The body carries **data sent to the server**, mainly with POST, PUT, and PATCH.

Example:

POST /employees

Content-Type: application/json

```
{  
  "employee_name": "Rahul Kumar",  
  "email": "rahul@example.com",  
  "phone": "9876543210",  
  "department_id": 1  
}
```

The API receives this data, validates it, processes it, and stores it in the database.

4.7 Complete Request Example

POST /employees

Content-Type: application/json

Accept: application/json

```
{  
  "employee_name": "Rahul Kumar",  
  "email": "rahul@example.com",  
  "department_id": 1  
}
```

Flow:

Client



HTTP Request



REST API



Validate Request



Process Data



Database

An HTTP request is the message sent by the client to our REST API. It contains the HTTP method, URL, headers, parameters and, when required, a request body. In our Employee Management API, these components together tell the backend which employee operation is required and what data should be processed.

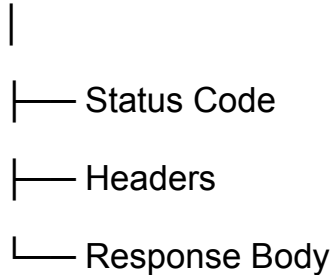
Request = What the client wants + the information required to perform it.

5. HTTP Response

An **HTTP Response** is the message returned by the server/API to the client after processing an HTTP request.

5.1 Response Structure

HTTP Response



5.2 Status Code

The status code tells the client **what happened with the request**.

Status	Meaning	Example
200 OK	Request successful	Employee retrieved
201 Created	Resource created	Employee added
204 No Content	Successful, no response body	Employee deleted
400 Bad Request	Invalid request	Incorrect input
404 Not Found	Resource doesn't exist	Employee ID not found

409 Conflict	Data conflict	Duplicate email
422 Unprocessable Entity	Validation failed	Invalid field value
500 Internal Server Error	Server-side failure	Unexpected backend error

5.3 Response Headers

Headers provide additional information about the response.

Example:

Content-Type: application/json

This tells the client that the response data is in JSON format.

5.4 Response Body

The response body contains the actual data returned by the API.

Example:

```
{  
  "employee_id": 101,  
  "employee_name": "Rahul Kumar",  
  "email": "rahul@example.com",  
  "department_id": 1  
}
```

For an error:

```
{
```

```
"detail": "Employee with ID 101 not found"
}
```

5.5 Complete Example

Client sends:

GET /employees/101

API processes the request and returns:

HTTP/1.1 200 OK

Content-Type: application/json

```
{
  "employee_id": 101,
  "employee_name": "Rahul Kumar",
  "email": "rahul@example.com"
}
```

5.6 Request → Response Flow

Client

|

| GET /employees/101

▼

REST API

|

| Find + Validate

▼

Database

|

| Employee Data



REST API

|

| 200 OK + JSON



Client

6. JSON Data Format

JSON (JavaScript Object Notation) is a lightweight, structured format used to exchange data between the client and REST API.

In our Employee Management API, JSON is primarily used for **request and response data**.

6.1 Why JSON?

JSON is:

- Easy for humans to read
- Easy for applications to parse
- Lightweight
- Language-independent
- Commonly used with REST APIs

6.2 JSON Object

An employee can be represented as:

{

"employee_id": 101,

"employee_name": "Rahul Kumar",

```
"email": "rahul@example.com",  
"department_id": 1  
}
```

Here:

employee_id → **key**

101 → **value**

employee_name → **key**

"Rahul Kumar" → **value**

6.3 Common JSON Data Types

Type	Example
String	"Rahul Kumar"
Number	101
Boolean	true
Null	null
Array	["IT", "HR"]
Object	{ "id": 101 }

Example:

```
{  
  "employee_id": 101,  
  "employee_name": "Rahul Kumar",  
  "active": true,  
  "phone": null,  
  "skills": ["Python", "SQL"]  
}
```

So the basic communication becomes:

Client

|

| JSON Request

▼

REST API

|

| Validate + Process

▼

Database

|

▼

REST API

|

| JSON Response



Client

JSON is the standard data representation I am using for communication between the client and Employee REST API. It represents employee information as key-value pairs and can be used in both request bodies and response bodies.

JSON = Structured data exchanged between client and API.

7. HTTP Headers

HTTP Headers are key-value pairs that provide **additional information about an HTTP request or response.**

They help the client and server understand **how the data should be interpreted and processed.**

7.1 Request Headers

Request headers are sent **from the client to the API.**

Example:

POST /employees

Content-Type: application/json

Accept: application/json

Authorization: Bearer <token>

Common headers:

Header	Purpose
--------	---------

Content-Type	Defines the format of request data
Accept	Defines the response format expected
Authorization	Sends authentication credentials
User-Agent	Identifies the client/application

7.2 Response Headers

Response headers are sent **from the API to the client**.

Example:

HTTP/1.1 200 OK

Content-Type: application/json

The client can use these headers to understand the response.

7.3 Content-Type

One of the most important headers for our API:

Content-Type: application/json

It tells the server:

The request body contains JSON data.

For example:

POST /employees

Content-Type: application/json

```
{  
  "employee_name": "Rahul Kumar",  
  "email": "rahul@example.com"  
}
```

7.4 Accept

Accept: application/json

This tells the server:

I expect the response in JSON format.

7.5 Authorization

For secured APIs, authentication information can be sent through:

Authorization: Bearer <token>

The server can use the token to determine whether the client is authorized to access the resource.

7.6 Headers in Our Employee API

A typical request could look like:

POST /employees

Content-Type: application/json

Accept: application/json

Authorization: Bearer <token>

```
{  
  "employee_name": "Rahul Kumar",
```

```
"email": "rahul@example.com",  
"department_id": 1  
}
```

HTTP headers carry metadata about an API request or response. In our Employee API, Content-Type tells the server that the request body is JSON, Accept indicates the expected response format, and Authorization can carry authentication information.

Headers = Metadata that helps the client and server communicate correctly.

6. URL (Uniform Resource Locator)

A **URL** identifies the resource that a client wants to access through an API. It acts as the **address of an API resource**.

URL identifies the resource, while the HTTP method defines the operation.

URL Components:-

https://api.example.com/v1/employees/101

Protocol	Host	Version	Resource	ID

- **Protocol** → Defines how communication takes place.
- **Host** → Identifies the server providing the API.
- **Version** → Helps manage different API versions.
- **Resource** → Identifies the type of data being accessed.
- **ID** → Identifies a specific resource.

Key Points

- A URL provides the **address of an API resource**.
- REST URLs generally represent **resources, not actions**.
- `/employees` represents the employee collection.
- `/employees/101` represents one specific employee.

- The **HTTP method** determines the operation performed on the resource.

REST API Examples

/api/employees

/api/departments

/api/leave-types

/api/leave-requests

For example:

GET /api/employees/101

This means the client wants to **retrieve employee 101**.

A URL is the address used to identify a resource in an API.

9. Query Parameters

Query parameters are optional values added to a URL to tell the API **how the client wants the resource data to be returned**.

Query parameters are optional values added to the URL to **filter, search, sort, or control the data returned by an API**.

They are written after **?** in the URL

They are mainly useful for **filtering, searching, sorting, and pagination**.

Example

Suppose our Employee API contains many employees:

GET /api/employees?location=Noida

This tells the API:

Return employees whose location is Noida.

Multiple Query Parameters

GET /api/employees?location=Noida&designation=Developer

Here:

- `location=Noida` → Filter by location
- `designation=Developer` → Filter by designation
- `&` → Separates multiple parameters

Different Practical Examples

1. Search

GET /api/employees?search=Shivam

→ Find employees matching **Shivam**.

2. Sorting

GET /api/employees?sort=joining_date

→ Return employees sorted by joining date.

3. Pagination

GET /api/employees?page=2&limit=5

→ Return the second page with 5 employees.

4. Filtering

GET /api/employees?status=ACTIVE

→ Return only active employees.

Key Points

- Query parameters are usually **optional**.
- They start after the `?` symbol.
- Multiple parameters are separated using `&`.
- They normally work with a **resource collection**.
- They do not usually identify one specific record.

Path vs Query Parameter

Path Parameter

/api/employees/105

→ Identifies employee 105

Query Parameter

/api/employees?status=ACTIVE

→ Filters the employee collection

Path parameter identifies a resource; query parameter controls the result set.

10. Path Parameters

A **path parameter** is a variable value included directly in the URL path to identify a **specific resource**.

A path parameter is a variable part of a URL used to identify a specific resource in a web application or API.

Example

GET /api/employees/205

Here:

/api/employees/205

└── Path Parameter

205 represents the **employee ID**.

The API interprets this as:

“Get the employee whose ID is 205.”

Different Examples

1. Get a specific employee

GET /api/employees/205

→ Employee with ID 205

2. Get a specific department

GET /api/departments/10

→ Department with ID 10

3. Get a specific leave request

GET /api/leave-requests/501

→ Leave request with ID 501

Path Parameter in API Design

A common REST pattern is:

/api/{resource}/{id}

For example:

/api/employees/205

/api/departments/10

/api/leave-requests/501

The {id} identifies the particular resource.

Key Points

- Written as part of the URL path.
- Used to identify a **specific resource**.
- Usually represents an ID or unique identifier.
- Commonly used with GET, PUT, PATCH, and DELETE.
- In FastAPI, it can be defined using {id} in the route.

11. Request Body

The **request body** contains the actual data that the client sends to the API for processing.

It is mainly used with methods such as **POST, PUT, and PATCH**.

The API receives this data and can then **validate it, apply business rules, and store it in the database**.

Different Request Body Examples

Create Employee — POST

```
{  
  "employee_name": "Aman Verma",  
  "email": "aman@example.com",  
  "department_id": 2  
}
```

Update Employee — PUT

```
{  
  "employee_name": "Aman Sharma",  
  "email": "aman@example.com",  
  "department_id": 3  
}
```

Partial Update — PATCH

```
{  
  "department_id": 4  
}
```

Here, only the department is being changed.

Key Points

- Contains data sent from **client** → **API**.
- Commonly used with **POST**, **PUT**, and **PATCH**.
- Usually formatted as **JSON** in REST APIs.
- The API should validate the body before processing it.
- Required and optional fields should be clearly defined.

The request body contains the actual data that the client sends to the API. In our Employee Management API, it is mainly used when creating or updating employees. The API receives the JSON body, validates the fields and then processes the data.

Request body = Data sent by the client for the API to process.

12. HTTP Methods

HTTP methods define the **operation that a client wants to perform on an API resource**.

In REST API design, the **URL identifies the resource**, while the **HTTP method defines the action** to be performed.

12.1 Core HTTP Methods

Method	Operation	Purpose
GET	Read	Retrieve existing data
POST	Create	Create a new resource

PUT	Replace	Replace the complete resource
PATCH	Partial Update	Modify specific fields
DELETE	Delete	Remove a resource

12.2 Employee API Examples

GET — Retrieve

GET /api/employees

Returns the employee collection.

GET /api/employees/205

Returns the employee with ID 205.

POST — Create

POST /api/employees

Request body:

```
{  
  "employee_name": "Aman Verma",  
  "email": "aman@example.com",  
  "department_id": 2  
}
```

Creates a **new employee resource**.

PUT — Replace

PUT /api/employees/205

```
{  
  "employee_name": "Aman Sharma",  
  "email": "aman@example.com",  
  "department_id": 3  
}
```

Used when the client wants to **replace the resource representation** with the supplied data.

PATCH — Partial Update

PATCH /api/employees/205

```
{  
  "department_id": 4  
}
```

Only the specified field is modified.

DELETE — Remove

DELETE /api/employees/205

Removes employee 205 from the resource.

Key Points

- **GET** → Read existing resource
- **POST** → Create a new resource
- **PUT** → Replace the resource
- **PATCH** → Update selected fields

- **DELETE** → Remove the resource
- The method should clearly communicate the intended operation.
- REST APIs use standard HTTP methods to make API behavior **consistent and predictable**.

URL identifies the resource → HTTP method defines the operation.

13. HTTP Status Codes

HTTP status codes are three-digit codes returned by the server to tell the client **what happened after processing the request**.

They help the client understand whether the request was **successful, invalid, unauthorized, not found, or failed because of a server problem**.

13.1 Status Code Categories

Category	Range	Meaning
1xx	100–199	Informational
2xx	200–299	Successful request
3xx	300–399	Redirection
4xx	400–499	Client-side error

5xx	500–599	Server-side error
------------	---------	-------------------

For our Employee API, **2xx**, **4xx**, and **5xx** are the most important categories.

13.2 Important API Status Codes

Code	Meaning	Employee API Example
200 OK	Request successful	Employee retrieved/updated
201 Created	Resource successfully created	New employee created
204 No Content	Successful request with no response body	Employee deleted
400 Bad Request	Invalid request	Malformed request
401 Unauthorized	Authentication required/failed	Missing or invalid credentials
403 Forbidden	Access not permitted	User lacks permission

404 Not Found	Resource doesn't exist	Employee ID not found
409 Conflict	Request conflicts with existing data	Duplicate email
422 Unprocessable Entity	Validation failed	Invalid employee data
500 Internal Server Error	Unexpected server-side error	Backend failure

13.3 Example: Successful Request

GET /api/employees/205

Response:

HTTP/1.1 200 OK

```
{  
  "employee_id": 205,  
  "employee_name": "Aman Verma"  
}
```

13.4 Example: Employee Not Found

GET /api/employees/999

Response:

HTTP/1.1 404 Not Found

```
{
```

```
"detail": "Employee not found"
}
```

13.5 Example: Duplicate Email

POST /api/employees

If the email already exists:

HTTP/1.1 409 Conflict

```
{
  "detail": "Employee with this email already exists"
}
```

Key Points

- Status codes communicate the **result of an API request**.
- **2xx** generally indicates success.
- **4xx** indicates a problem with the client's request or access.
- **5xx** indicates a server-side failure.
- Using appropriate status codes makes an API easier for clients to understand and handle.

Status code = Clear indication of what happened to the request.

14. Employee REST API Design

The Employee REST API provides standardized endpoints to **retrieve, create, update, and delete employee records**.

The API uses JSON for request/response data and appropriate HTTP status codes to communicate the result of each operation.

1. GET /employees

Purpose: Retrieve the list of employees available in the system.

Request JSON: Not required.

Example:

GET /employees

Response JSON:

```
{
  "employees": [
    {
      "employee_id": 101,
      "employee_name": "Arjun Mehta",
      "email": "arjun.mehta@example.com",
      "phone": "9876543210",
      "department_id": 1,
      "status": "ACTIVE"
    },
    {
      "employee_id": 102,
      "employee_name": "Neha Sharma",
      "email": "neha.sharma@example.com",
      "phone": "9812345678",
      "department_id": 2,
      "status": "ACTIVE"
    }
  ]
}
```

```
}
```

Success: 200 OK

Errors:

- 500 Internal Server Error → Unexpected server/database failure.

Validation: No request-body validation is required because the endpoint only retrieves data.

2. GET /employees/{id}

Purpose: Retrieve the details of one specific employee.

Example:

GET /employees/102

Request JSON: Not required.

Response JSON:

```
{  
  "employee_id": 102,  
  "employee_name": "Neha Sharma",  
  "email": "neha.sharma@example.com",  
  "phone": "9812345678",  
  "joining_date": "2025-07-15",  
  "department_id": 2,  
  "manager_id": 101,  
  "status": "ACTIVE"  
}
```

Success: 200 OK

Errors:

- **404 Not Found** → Employee with the given ID does not exist.
- **422 Unprocessable Entity** → ID format is invalid.

Validation:

- **id** must be a valid integer.
- The employee must exist in the database.

3. POST /employees

Purpose: Create a new employee record.

Request JSON:

```
{  
  "employee_name": "Kabir Malhotra",  
  "email": "kabir.malhotra@example.com",  
  "phone": "9898765432",  
  "joining_date": "2026-09-23",  
  "department_id": 1,  
  "manager_id": 101,  
  "status": "ACTIVE"  
}
```

Response JSON:

```
{  
  "employee_id": 103,  
  "employee_name": "Kabir Malhotra",  
  "email": "kabir.malhotra@example.com",
```

```
"phone": "9898765432",  
"joining_date": "2026-09-23",  
"department_id": 1,  
"manager_id": 101,  
"status": "ACTIVE"  
}
```

Success: 201 Created

Errors:

- 400 Bad Request → Request structure is invalid.
- 409 Conflict → Email already exists.
- 422 Unprocessable Entity → One or more fields failed validation.

Validation:

- `employee_name` is required.
- `email` is required and must have a valid format.
- `email` must be unique.
- `joining_date` is required.
- `department_id` must refer to an existing department.
- `manager_id`, when provided, must refer to a valid employee.
- Employee cannot be assigned as their own manager.
- `status` must contain an allowed value.

4. PUT `/employees/{id}`

Purpose: Replace the existing employee information with the supplied data.

Example:

PUT /employees/103

Request JSON:

```
{
```

```
"employee_name": "Kabir Malhotra",  
"email": "kabir.malhotra@company.com",  
"phone": "9800123456",  
"joining_date": "2026-09-23",  
"department_id": 3,  
"manager_id": 102,  
"status": "ACTIVE"  
}
```

Response JSON:

```
{  
  "employee_id": 103,  
  "employee_name": "Kabir Malhotra",  
  "email": "kabir.malhotra@company.com",  
  "phone": "9800123456",  
  "joining_date": "2026-09-23",  
  "department_id": 3,  
  "manager_id": 102,  
  "status": "ACTIVE"  
}
```

Success: 200 OK

Errors:

- 404 Not Found → Employee does not exist.
- 409 Conflict → Email is already associated with another employee.
- 422 Unprocessable Entity → Supplied data is invalid.

Validation:

- **id** must be a valid integer.
- Required employee fields must be provided.
- Email must have a valid format and remain unique.
- Department must exist.
- Manager must exist when provided.
- Employee cannot be their own manager.
- Status must be a permitted value.

5. DELETE **/employees/{id}**

Purpose: Remove a specific employee record from the system.

Example:

DELETE /employees/103

Request JSON: Not required.

Response: No response body.

Success: 204 No Content

Errors:

- 404 Not Found → Employee does not exist.
- 422 Unprocessable Entity → Employee ID is invalid.

Validation:

- **id** must be a valid integer.
- The employee must exist before deletion.

Final API Table:-

Method	Endpoint	Request Body	Status	Main Errors
--------	----------	--------------	--------	-------------

cases

GET	/employees	None	200	500
GET	/employees/{id}	None	200	404, 422
POST	/employees	Required	200, 201, 422	
PUT	/employees/{id}	Required	200	404, 409, 422
DELETE	/employees/{id}	None	204	404, 422

Standard API Errors:-

Status	Meaning	Example
--------	---------	---------

400	Bad Request	Invalid request structure
404	Not Found	Employee does not exist
409	Conflict	Duplicate employee email
422	Validation Error	Invalid field or ID
500	Internal Server Error	Unexpected backend/database failure